

Relazione Programma di Natale

La vista utente

La parte con cui ho deciso di iniziare è la parte grafica in quanto il primo problema che mi è saltato in mente leggendo la consegna era la grandezza della tabella e come creare i quadratoni che formano essa, su c++. Ho deciso di risolvere creando una tabella di dimensione fissa 20x20 in cui ogni quadratone di essa fosse formato da tre elementi: la parentesi quadra sinistra, un elemento centrale e la parentesi quadra destra ([elemento]). L'elemento centrale viene visualizzato in output come uno spazio, se l'elemento è morto, altrimenti come un asterisco invisibile all'utente in quanto sottolineato di rosso insieme alle parentesi ad esso adiacenti, in modo da formare dei quadratoni rossi. Questa tabella viene posizionata a sinistra alla vista dell'utente. In alto viene posizionato un titolo molto semplice e a sinistra un menu e dei contatori di vita e generazione. La tabella e i contatori vengono aggiornati spesso mentre il titolo resta fisso e il menu varia raramente per dare quelle poche interazioni all'utente.

La logica interna

Per far funzionare la visualizzazione della tabella ho deciso di utilizzare una matrice booleana 20x20. Inizialmente la matrice è completamente false in ogni suo valore in modo da visualizzare la tabella vuota all'utente, ma poi, partito il gioco, viene caricata di valori casuali in cui ci sono più probabilità che una cella sia "morta" (false) in modo da non avere in output una tabella completamente rossa. Essendo che ogni cella ha 8 celle adiacenti, ho deciso di analizzare cella per cella della matrice lavorando per righe, andando a studiare, per ognuna di esse, il numero totale di celle adiacenti "vive" a ogni elemento della matrice (tranne per casi speciali). Questo procedimento descritto prima non funziona in tutti i casi (in quanto ho utilizzato un mondo limitato) infatti per le celle agli estremi della tabella e per le celle posizionate ai lati della tabella ho dovuto sviluppare degli algoritmi che mi andassero a leggere solo le celle della matrice realmente esistenti. Per fare un esempio l'estremo della tabella in alto a sinistra non analizza la parte alta delle celle adiacenti e nemmeno il lato sinistro. Fatte le varie analisi con la matrice principale, in base al numero di celle adiacenti "vive", il programma seguendo le regole del gioco della vita carica dei valori in una matrice temporanea e finito di caricarla li copia nella principale in modo da non corrompere le analisi in corso e successivamente aggiorna l'output. Questo processo lo esegue ogni mezzo secondo.

Come finisce?

Uno dei problemi più grandi che mi si è riscontrato solamente alla fine è come far finire il programma. È stato un grosso problema in quanto, utilizzando uno sleep e per come ho disposto il programma, non potevo richiedere un input durante l'esecuzione senza dover per forza bloccare il programma. Analizzando l'output del

programma ho inoltre notato che si creano a volte delle situazioni di periodicità (es. 4 passaggi ripetuti all'infinito che tornano sempre al punto di partenza) casuali e con potenzialmente infinite combinazioni e quindi difficilmente risolvibili. Osservando ancora successivamente ho notato delle situazioni di stabilità infinita facilmente risolvibile. Per ovviare a questi problemi ho deciso di far chiudere il programma se si presenta una di queste 3 situazioni: se non resta più vita cioè se tutte le celle della matrice principale hanno valore false (tranne nella fase di pre-gioco), se ho un caso in cui la matrice della generazione precedente ha gli stessi valori della generazione attuale (situazione di stabilità) o se arrivo alla generazione 300, in modo da bloccare le generazioni periodiche e imprevedibili.